BORG AL ARAB TECHNOLOGICAL UNIVERSITY

CODE:

PROGRAMMING IN C

First Year –Information Technology Program Spring 2026

LECTURE 9

UNDERSTANDING STRING HANDLING IN C

INTRODUCTION TO STRINGS

- **Definition:** A string is a sequence of characters terminated by a null character (\0).
- **Data Type:** Strings in C are represented as arrays of characters

```
char str[20] = "Hello, World!";
```

DECLARING STRINGS

- Static Declaration:

```
char greeting[20] = "Hello!";
```

- Dynamic Declaration:

```
char *name = (char*)malloc(50 * sizeof(char));
```

STRING INITIALIZATION

Without Specified Size:

```
char message[] = "Welcome";
```

```
char status[] = "Active"; // Size is auto-determined
```

PRINT A STRING

- Note that you have to use double quotes ("").
- To output the string, you can use the printf() function together with the format specifier %s to tell C that we are now working with strings

```c
#include <stdio.h>

int main() {
  char greetings[] = 'Hello World!';
  printf("%s", greetings);

  return 0;
}
```

```
prog.c: In function ‘main’:
prog.c:4:22: warning: character constant too long for its type
    4 |   char greetings[] = 'Hello World!';
      |                      ^~~~~~~~~~~~~~
prog.c:4:22: error: invalid initializer
```

```c
#include <stdio.h>

int main() {
  char greetings[] = "Hello World!";
  printf("%s", greetings);

  return 0;
}
```

```
Hello World!
```

ACCESS STRINGS

- Since strings are actually arrays in C, you can access a string by referring to its index number inside square brackets [].
- This example prints the **first character (0)** in **greetings**:

```c
#include <stdio.h>

int main() {
  char greetings[] = "Hello World!";
  printf("%c", greetings[0]);

  return 0;
}
```

```
H
```

- Note that we have to use the **%c** format specifier to print a **single character**.

```c
#include <stdio.h>

int main() {
  char greetings[] = "Hello World!";
  printf("%c", greetings[0]);

  return 0;
}
```

```
H
```

MODIFY STRINGS

- To change the value of a specific character in a string, refer to the index number, and use **single quotes**:

```c
#include <stdio.h>

int main() {
  char greetings[] = "Hello World!";
  greetings[0] = 'J';
  printf("%s", greetings);

  return 0;
}
```

```
Jello World!
```

LOOP THROUGH A STRING

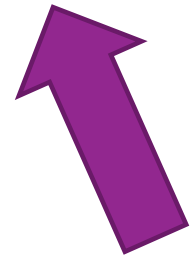
You can also loop through the characters of a string, using a for loop:

```c
#include <stdio.h>

int main() {
  char carName[] = "Volvo";
  int i;

  for (i = 0; i < 5; ++i) {
    printf("%c\n", carName[i]);
  }

  return 0;
}
```



```
V
o
l
v
o
```

- you can also use the *sizeof formula* (instead of manually write the size of the array in the loop condition (i < 5) :

```c
#include <stdio.h>

int main() {
  char carName[] = "Volvo";  // Initializing the array with the string "Volvo"
  int length = sizeof(carName);  // Get the size of the array, which includes the null ter
  int i;

  // Print the size of the carName array (including the null terminator)
  printf("sizeof(carName) is %i \n", sizeof(carName));

  // Loop through each character in carName array and print it
  for (i = 0; i < length; ++i) {
    printf("%c\n", carName[i]);
  }

  return 0;
}
```

ANOTHER WAY OF CREATING STRINGS

```c
#include <stdio.h>

int main() {
  char greetings[] = {'H', 'e', 'l', 'l', 'o', ' ', 'W', 'o', 'r', 'l', 'd', '!', '\0'};
  char greetings2[] = "Hello World!";

  printf("%lu\n", sizeof(greetings));
  printf("%lu\n", sizeof(greetings2));

  return 0;
}
```

```
13
13
```

Common String Functions:

- strlen() - Returns the length of a string (**excluding the null character**)
- strcpy() - Copies one string to another.
- strcat() - Concatenates two strings.
- strcmp() - Compares two strings.

EXAMPLE: STRING LENGTH

```
1  #include <stdio.h>
2  #include <string.h>
3
4  int main() {
5      char str[] = "Hello, World!";
6      printf("Length: %lu\n", strlen(str));
7      return 0;
8  }
```

Print output (drag lower right corner to resize)

Stack

main

str

	0	1	2	3	4	5	6	7	8	9	10	11	12	13
array	char ?	char ?	char ?	char ?	char ?	char ?	char ?	char ?	char ?	char ?	char ?	char ?	char ?	char ?

Note: ? refers to an uninitialized value

```
1  #include <stdio.h>
2  #include <string.h>
3
4  int main() {
5      char str[] = "Hello, World!";
6      printf("Length: %lu\n", strlen(str));
7      return 0;
8  }
```

Print output (drag lower right corner to resize)

```
Length: 13
```

Stack

main

str

	0	1	2	3	4	5	6	7	8	9	10	11	12	13
array	char 'H'	char 'e'	char 'l'	char 'l'	char 'o'	char ','	char ' '	char 'W'	char 'o'	char 'r'	char 'l'	char 'd'	char '!'	char '\0'

```c
char alphabet[] = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
printf("%d", strlen(alphabet));   // 26
printf("%d", sizeof(alphabet));   // 27
```

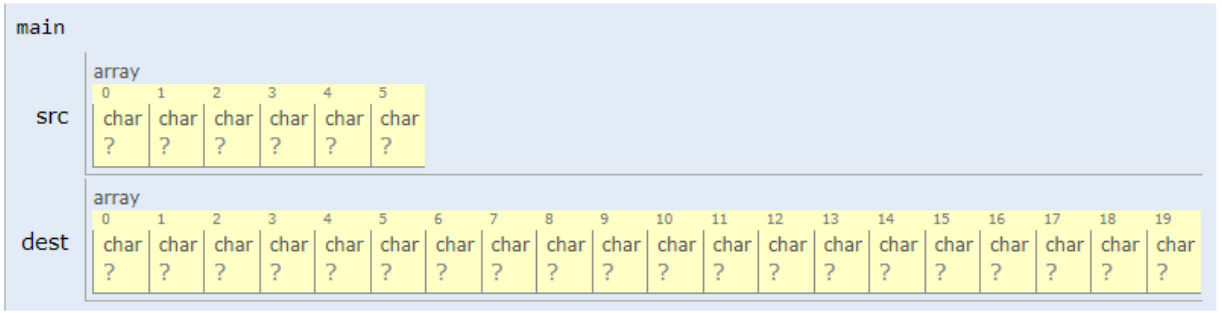
```c
char alphabet[50] = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
printf("%d", strlen(alphabet));   // 26
printf("%d", sizeof(alphabet));   // 50
```

EXAMPLE: STRING COPY

```
1  #include <stdio.h>
2  #include <string.h>
3
4  int main() {
5      char src[] = "Hello";
6      char dest[20];
7      strcpy(dest, src);
8      printf("Copied String: %s\n", dest);
9      return 0;
10 }
```

Print output (drag lower right corner to resize)

Stack

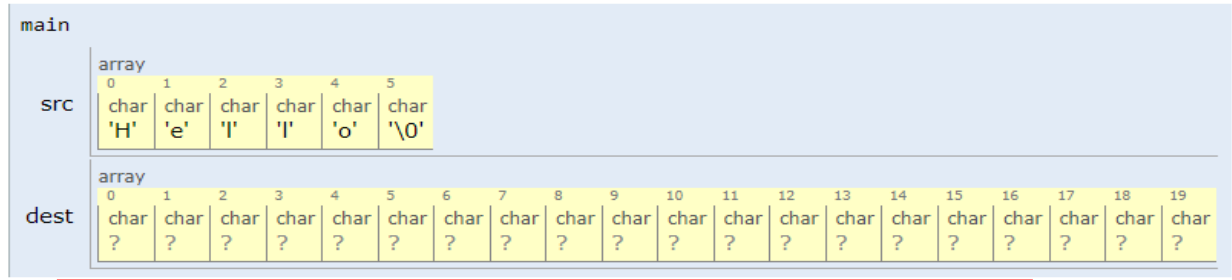


Note: ? refers to an uninitialized value

```
7      strcpy(dest, src);
```

Print output (drag lower right corner to resize)

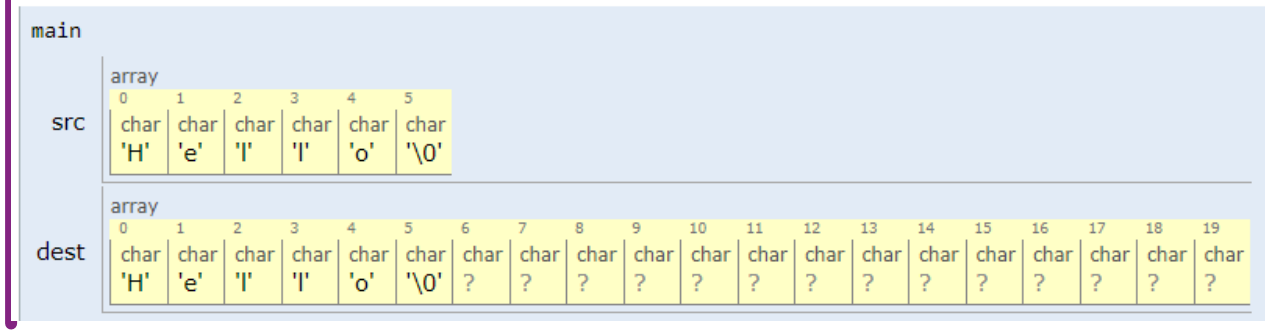
Stack




```
9      return 0;
10 }
```

Copied String: Hello

Stack



EXAMPLE: STRING CONCATENATE

```
1  #include <stdio.h>
2  #include <string.h>
3
4  int main() {
5      char str1[20] = "Hello";
6      char str2[] = " World!";
7      strcat(str1, str2);
8      printf("Concatenated String: %s\n", str1);
9      return 0;
10 }
```

Print output (drag lower right corner to resize)

Stack

main

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
str1	char ?	char ?	char ?	char ?	char ?	char ?	char ?	char ?	char ?	char ?	char ?	char ?	char ?	char ?	char ?	char ?	char ?	char ?	char ?	char ?

	0	1	2	3	4	5	6	7
str2	char ?	char ?	char ?	char ?	char ?	char ?	char ?	char ?

Note: ? refers to an uninitialized value

```
7      strcat(str1, str2);
```

Print output (drag lower right corner to resize)

Stack

main

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
str1	char 'H'	char 'e'	char 'l'	char 'l'	char 'o'	char '\0'	char '\0'	char '\0'	char '\0'	char '\0'	char '\0'	char '\0'	char '\0'	char '\0'	char '\0'	char '\0'	char '\0'	char '\0'	char '\0'	char '\0'

	0	1	2	3	4	5	6	7
str2	char ' '	char 'W'	char 'o'	char 'r'	char 'l'	char 'd'	char '!'	char '\0'

```
8      printf("Concatenated String: %s\n", str1);
9      return 0;
```

```
Concatenated String: Hello World!
```

Stack

main

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
str1	char 'H'	char 'e'	char 'l'	char 'l'	char 'o'	char ' '	char 'W'	char 'o'	char 'r'	char 'l'	char 'd'	char '!'	char '\0'	char '\0'	char '\0'	char '\0'	char '\0'	char '\0'	char '\0'	char '\0'

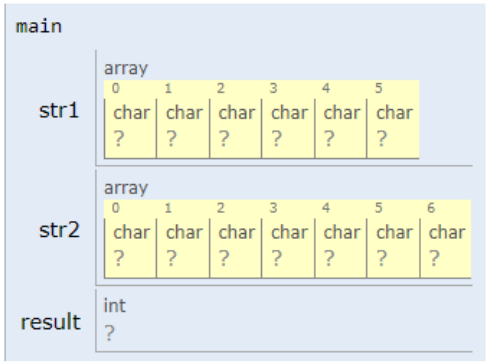
	0	1	2	3	4	5	6	7
str2	char ' '	char 'W'	char 'o'	char 'r'	char 'l'	char 'd'	char '!'	char '\0'

EXAMPLE: STRING COMPARISON

```
1  #include <stdio.h>
2  #include <string.h>
3
4  int main() {
5      char str1[] = "apple";
6      char str2[] = "banana";
7      int result = strcmp(str1, str2);
8      printf("Comparison Result: %d\n", result);
9      return 0;
10 }
```

Print output (drag lower right corner to resize)

Stack

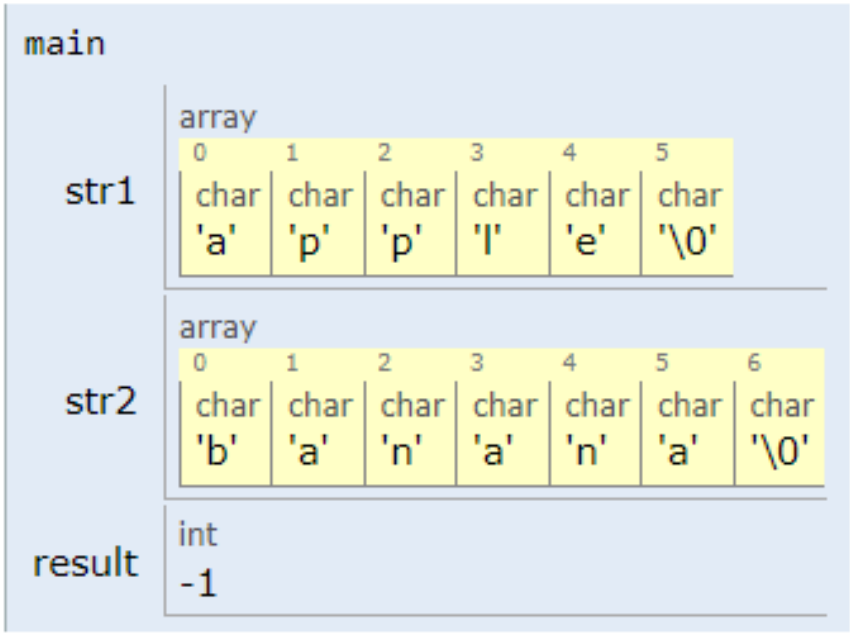



```
8      printf("Comparison Result: %d\n", result);
9      return 0;
```

Print output (drag lower right corner to resize)

```
Comparison Result: -1
```

Stack



WHAT IS OUT OF BOUNDS INDEX IN AN ARRAY - C LANGUAGE?

- For an array of size n, valid indices range from **0 to n-1**.
- If you declare an array with 4 elements, the valid indices are 0, 1, 2, and 3.
- Accessing index 4 or any higher value results in undefined behavior.
- The C compiler **does not perform bounds checking**, so out-of-bounds access **compiles successfully but produces unpredictable results.**

```c
#include <stdio.h>

int main() {
  char carName[] = "Volvo";
  carName[10]='A';    // Invalid assignment (out of bounds)
  printf("%c\n", carName[10]);// Invalid print (out of bounds)
  return 0;
}
```

WHY THIS IS DANGEROUS

- While the above example might show the expected value ‘A ‘, this behavior is **undefined**.
- The program might
  - Display garbage values
  - Crash with a segmentation fault
  - Overwrite other variables in memory
  - Appear to work correctly but cause bugs

IMPORTANT POINTS TO REMEMBER

- Strings are null-terminated arrays of characters.
- Always allocate sufficient memory for strings.
- Use string handling functions from <string.h> for safety and ease.
- **Initialization:** Properly initialize strings. Uninitialized strings may contain garbage values, leading to unpredictable behavior.

- **Happy Coding**